Student Performance & Attendance Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410002	26102000121	BWU/MCA/26/003	Annesha Chowdhury

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



NOVEMBER 2026

Student Performance & Attendance Analyzer

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410002	26102000121	BWU/MCA/26/003	Annesha Chowdhury

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

[COMPUTATIONAL SCIENCES]

BONAFIDE CERTIFICATE

Certified that this project report **"Student Performance & Attendance Analyzer"** is the bonafide work **of "Annesha Chowdhury"** who carried out the project work under my supervision.

_ _ _ _ _ _ _ _ _ _ _ _	_ _ _ _ _ _ _ _ _ _ _ _
SIGNATURE	**SIGNATURE**
DR. Jayanta Aich	MR. Rahul Kumar Ghosh
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Computational Sciences	Assistant Professor
Brainware University	Computational Sciences

ABSTRACT

The increasing volume of academic data in educational institutions has made traditional manual methods of tracking student performance and attendance both time-consuming and error-prone. Teachers and academic administrators often face difficulties in quickly identifying students with low attendance, poor academic results, or those at risk of failing, as these processes typically involve repetitive calculations, spreadsheet management, and manual report generation. To address these challenges, this project presents the design and development of a **Student Performance & Attendance Analyzer** — a Python-based desktop application that automates the analysis of student academic and attendance data.

The system is designed to read structured student records from a CSV file containing essential information such as student identifiers, subject marks, and attendance details. Using the powerful data-handling capabilities of the **pandas** library, the application loads, validates, and processes the data efficiently. Custom analysis modules then compute critical performance metrics for each student, including average marks, attendance percentage, grade assignment based on predefined thresholds, and pass/fail status (considering both academic performance and a minimum attendance requirement, commonly set at 75%). At the class level, the system generates comprehensive statistical summaries such as overall average performance, highest and lowest scorers, grade distribution, and lists of students requiring special attention due to low attendance or poor results.

Visualization forms a core component of the system. Through the **matplotlib** library, the application produces clear and insightful graphical representations, including bar charts for performance comparison, histograms for attendance distribution, pie charts for grade breakdown, and other relevant plots. These visual outputs help educators quickly interpret trends and make informed decisions without the need for complex data analysis tools.

The software follows a clean modular architecture consisting of two primary Python modules:

- **analyzer.py** – Contains core logic for metric calculation, filtering, and statistical aggregation.
- **main.py** – Serves as the entry point, providing a simple command-line interface (CLI) that allows users to load data, run analysis, view statistics, generate charts, and exit the program.

This modular design enhances code readability, maintainability, and scalability, making the system easy to understand for beginners while remaining flexible for future enhancements. The application requires no advanced programming knowledge from the end user; once the CSV file is prepared, the system can be operated through a straightforward menu-driven interface.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	5
5	System Flow	6
6	Proposed Design	13
7	Experimental Result	15
8	Future Scope	18
9	Conclusion	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	7
5.3	Data Flow Diagram Level 1	8
5.4	Use Case Diagram	9
5.5	Activity Diagram	10
5.6	Sequence Diagram	11
5.7	Flow Chart	12
7.1	Data Loading Stage	15
7.2	Performance Analysis	15
7.3	Attendance Analysis	16
7.4	Bar Chart Generation	16
7.5	Line Graph Generation	17
7.6	Histogram Generation	17

CHAPTER 1: INTRODUCTION

In the modern educational landscape, tracking student academic performance and attendance has become increasingly complex due to the growing volume of data generated every semester. Traditional methods that rely on manual registers, handwritten attendance sheets, and basic spreadsheets are not only time-consuming but also prone to human errors. Teachers and academic administrators often spend considerable effort calculating averages, determining pass/fail status, identifying students with low attendance, and preparing summary reports. These repetitive tasks leave little time for meaningful academic intervention or personalized student support.
With the rapid advancement of programming languages and data analysis libraries, it is now possible to automate large portions of this process. Python, with its rich ecosystem of libraries such as **pandas** for data manipulation and **matplotlib** for visualization, offers a powerful yet accessible platform for building practical educational tools. This project, titled **Student Performance & Attendance Analyzer**, leverages these capabilities to create a simple, modular, and effective system that transforms raw student data into actionable insights.

The core idea of the project is straightforward: a user provides a structured CSV file containing student records (including identifiers, subject marks, and attendance information). The system then automatically loads the data, validates the records, calculates essential metrics such as average marks, attendance percentage, grades, and pass/fail status, identifies students who need special attention, and generates both textual statistics and clear graphical visualizations. This automated workflow eliminates the need for manual calculations and makes performance monitoring faster, more accurate, and more insightful.

This modular design not only improves code readability and maintainability but also makes the system easy for beginners to understand and extend. The project demonstrates how data processing, statistical analysis, and visualization techniques can be combined into a lightweight educational analytics tool suitable for teachers, academic administrators, and students learning Python programming.

By automating routine analytical tasks and presenting results in both numerical and visual formats, the Student Performance & Attendance Analyzer aims to reduce the administrative burden on educators while enabling quicker identification of academic and attendance-related issues. Ultimately, the project showcases the practical application of Python in solving real-world problems in the education domain.

CHAPTER 2: OBJECTIVE

The primary objective of this project is to design and develop a practical, easy-to-use **Python-based Student Performance & Attendance Analyzer** that automates the process of evaluating student academic performance and attendance from structured data. In most educational institutions, teachers and administrators still rely on manual calculations or basic spreadsheets to determine average marks, attendance percentages, grades, and pass/fail status. These traditional methods are time-consuming, prone to errors, and make it difficult to quickly identify students who require special attention. This project aims to overcome these limitations by providing an automated, modular, and user-friendly solution.
The system is designed to achieve the following specific goals:

1. **Efficient Data Processing**
   Enable seamless loading and validation of student records from a CSV file containing essential information such as student ID, name, subject marks, and attendance details.
2. **Accurate Metric Calculation**
   Automatically compute important individual and class-level metrics, including:

- Average marks / overall performance score
- Attendance percentage
- Grade assignment based on predefined mark ranges
- Pass/Fail status (considering both academic performance and a minimum attendance threshold, typically 75%)

3. **Identification of At-Risk Students**
   Detect and highlight students with low attendance or poor academic performance so that timely academic intervention can be planned.
4. **Statistical Summaries**
   Generate meaningful class-level statistics such as overall average, highest and lowest performers, grade distribution, and attendance trends.
5. **Visual Representation of Data**
   Produce clear and informative charts (bar charts, histograms, pie charts, etc.) using matplotlib to help users quickly understand performance patterns and attendance distribution.
6. **User-Friendly Interface**
   Provide a simple command-line interface that allows users to load data, run analysis, view statistics, generate charts, and exit the program without requiring advanced programming knowledge.
7. **Modular and Maintainable Design**
   Implement the system using a clean modular architecture (student.py, analysis.py, charts.py, and main.py) so that the code remains easy to understand, modify, and extend in the future.

By combining the data-handling power of **pandas**, custom analysis logic, and the visualization capabilities of **matplotlib**, the project aims to create a complete end-to-end pipeline for student performance and attendance analysis.

CHAPTER 3: PLANNING

The planning phase outlines the step-by-step approach used to design, organize, and implement the Student Performance & Attendance Analyzer. Proper planning ensures that the workflow remains efficient, modular, and easy for beginners to understand and use.

1. Requirement Identification

- Understand the need for a simple Python tool to analyze student marks and attendance from CSV data.
- Identify the target users (teachers, students, academic administrators, beginners).
- Define the core requirement: CSV data → analysis of performance & attendance → statistics + charts.

2. System Design Planning

- Decide to use pandas for data handling and matplotlib for visualization.
- Choose a modular structure with separate files for Student class, analysis logic, chart generation, and main program.
- Plan the workflow from data loading to final report and charts.
- Select supporting libraries listed in requirements.txt.

3. Pipeline Structure

Design a sequential pipeline with the following steps:

- Data loading from students.csv
- Student object creation / data validation
- Performance metric calculation
- Attendance percentage calculation
- Grade assignment and pass/fail determination
- Statistical summaries
- Chart generation
- Result display

4. File & Module Planning

- analyzer.py – Core calculation and analysis functions
- main.py – Entry point and user interaction / menu
- students.csv – Sample input data
- requirements.txt – Dependencies

5.Code Structure Planning

- Break the code into clear, reusable sections:
- Data loading and validation
- Metric calculation functions
- Filtering of low-attendance / low-performance students
- Chart plotting functions
- Main menu / command-line interface

6.Testing & Validation Plan

- Test with sample CSV data.
- Verify correct average, percentage, and grade calculations.
- Check chart generation for different metrics.
- Validate handling of edge cases (missing data, zero attendance, etc.).

7.Scalability & Improvement Planning

Plan for expandable features such as:

- Support for multiple subjects or semester data
- Export of reports to PDF or Excel
- Interactive GUI (Tkinter or Streamlit)
- Predictive analysis using simple machine learning
- Database integration instead of CSV

CHAPTER 4: REQUIREMENT ANALYSIS

❑ **Functional Requirements**

1.Data Input

- The system must read student records from a CSV file (students.csv).
- Each record typically includes student identifiers, marks, and attendance information.
- Input is handled through Python file reading and pandas DataFrame operations.

2.User Interaction

- A simple command-line interface or menu-driven program in main.py.
- Options to load data, run analysis, view statistics, generate charts, and exit.
- Users should not need to write code to obtain results.

3.Model and Framework Requirements

The project depends on the following Python libraries (as listed in requirements.txt):
- pandas (data loading and manipulation)
- matplotlib (chart generation)
- Other standard libraries (os, csv, etc.) as needed
- Custom modules: student.py, analysis.py, charts.py

4.Analysis Requirements

For every student record:
- Calculate average marks / total performance score.
- Calculate attendance percentage.
- Assign grades based on marks.
- Determine pass/fail status (considering both marks and minimum attendance threshold, typically 75%).
- Identify students with low attendance or poor performance.

5.Visualization Requirement

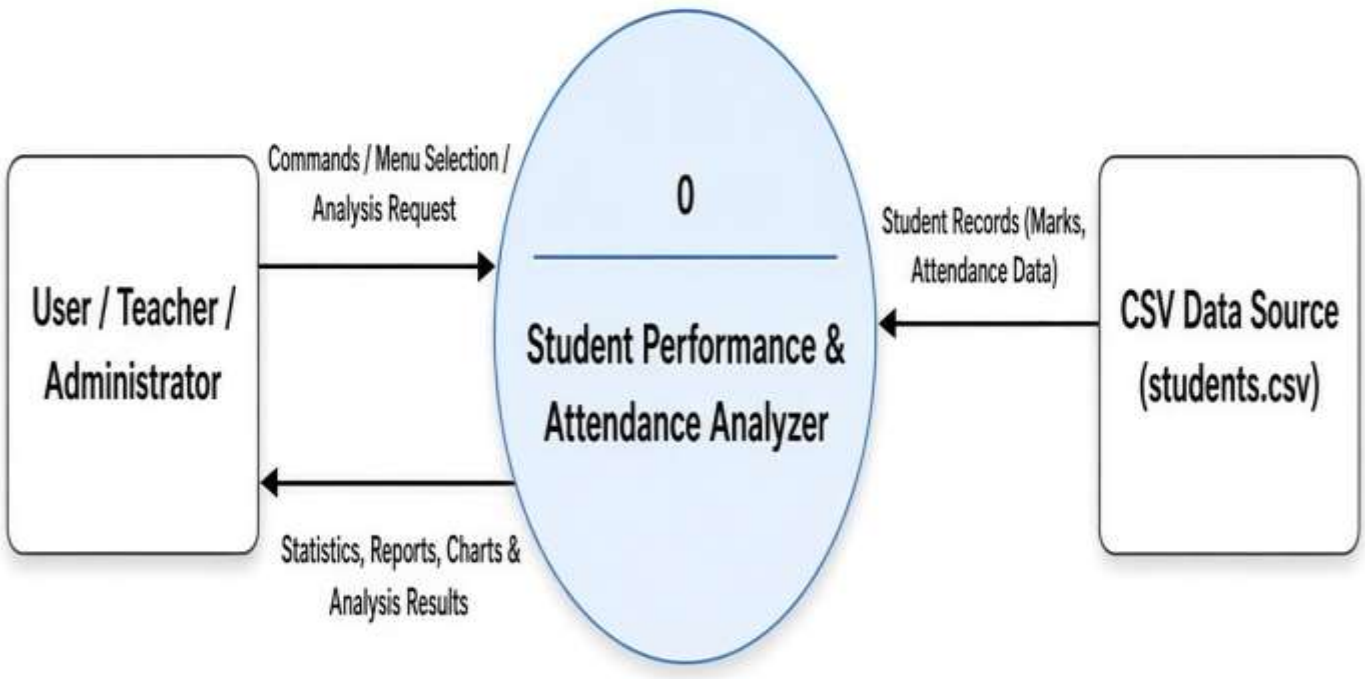
- Generate bar charts, pie charts, or other suitable plots for grade distribution, attendance ranges, performance comparison, etc.
- Charts should be saved or displayed clearly.

6.Output Organization

The system must:
- Display textual summaries and statistics on the console.
- Generate and save chart images.
- Provide a clear final report of class-level and individual insights.

CHAPTER 5: SYSTEM FLOW

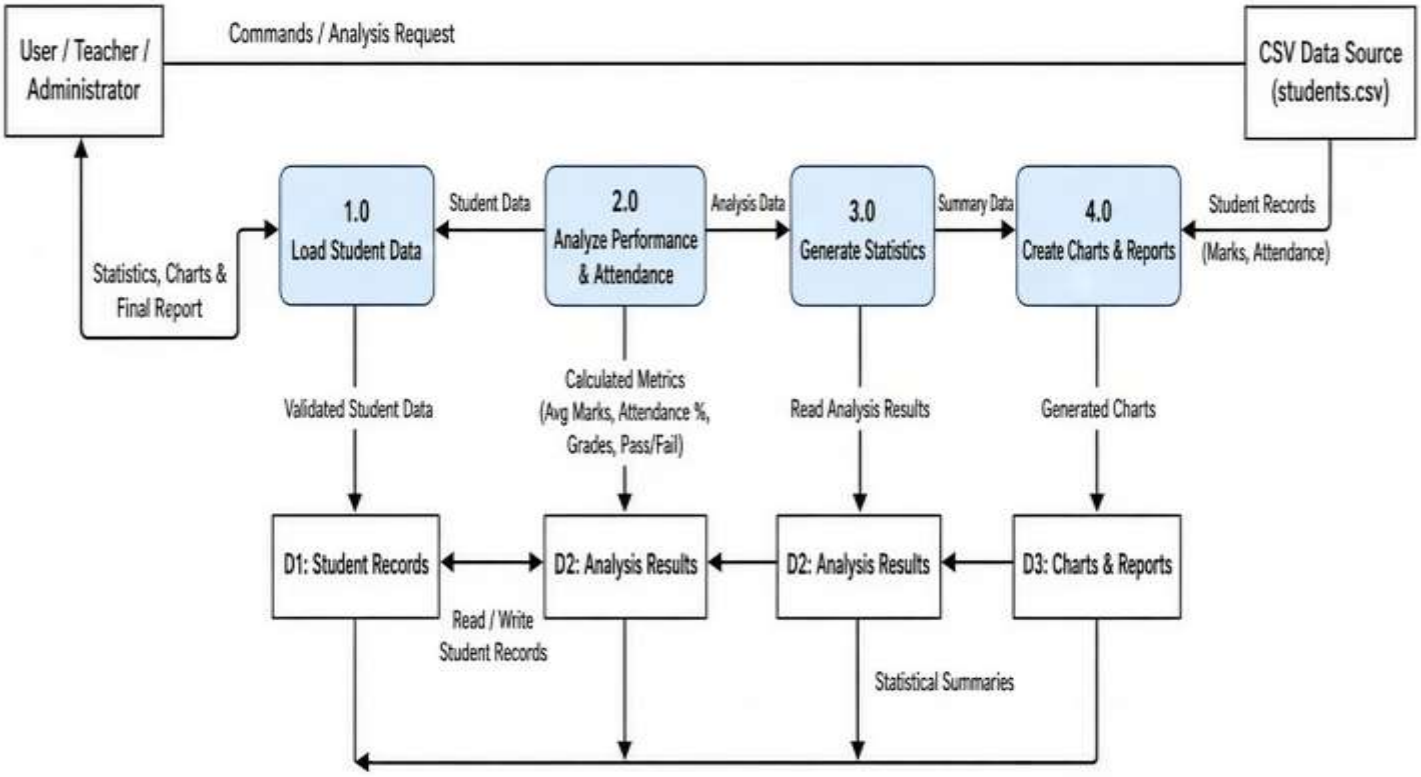
Context Level Diagram (CLD):



The generated Context Level Diagram shows the Student Performance & Attendance Analyzer as a single central process. This high-level diagram clearly defines the system boundary and its interaction with the outside world.

Fig 5.1: Context Level Diagram

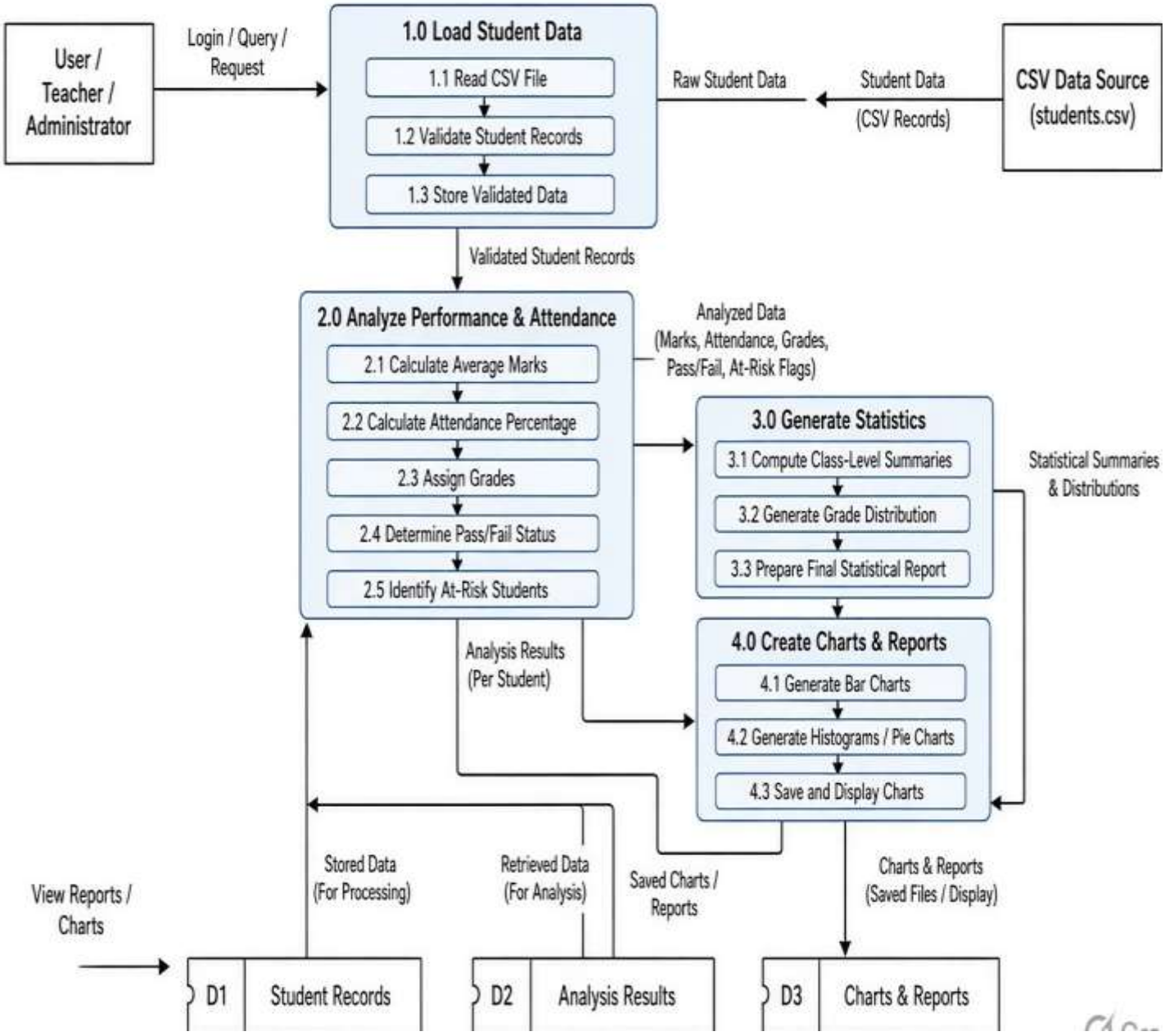
Data Flow Diagrams (DFD):



This diagram provides a clear overview of how data moves between the main functional modules.It is useful for understanding the overall structure and major components of the system.

Fig 5.2: Data Flow Diagram Level 0

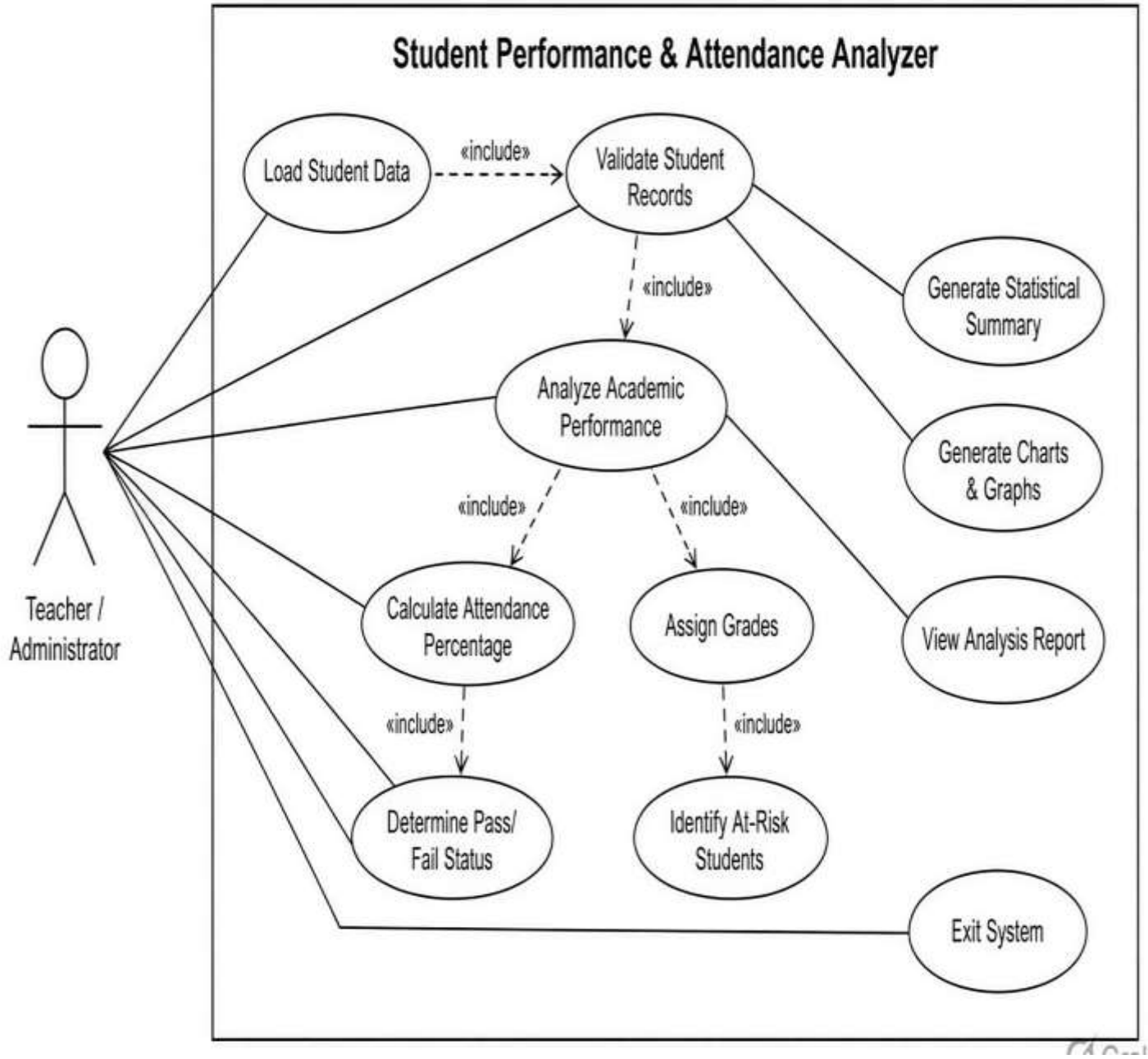
Data Flow Diagrams (DFD):



The diagram above expands the major processes of Level 0 into detailed sub-processes.This Level 1 DFD provides a clearer view of the internal data transformations and the detailed flow of information within the system. It helps developers and users understand the exact sequence of data transformations inside the system.

Fig 5.3: Data Flow Diagram Level 1

Use Case Diagram:



It depicts the interactions between the Teacher/Administrator and the system's key functionalities such as loading data, analyzing performance, and generating reports. This diagram highlights the main features available to the primary user of the system.It clearly shows the relationship between the actor and the different use cases supported by the application.

Fig 5.4: Use Case Diagram

Activity Diagram:

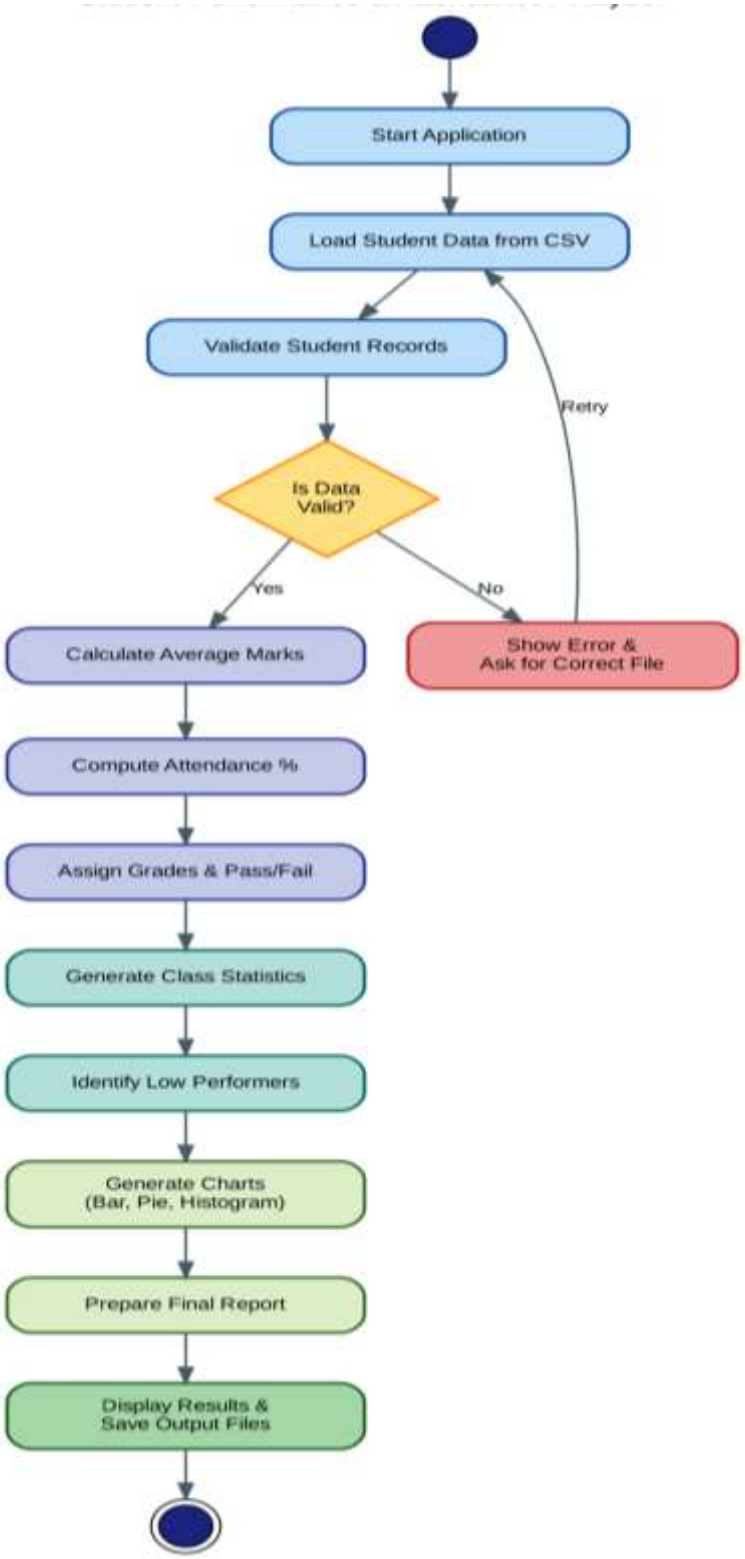


Fig 5.5: Activity Diagram

It represents the step-by-step workflow of the system from starting the application to displaying the final results, including decision points for data validation. This diagram clearly shows the complete workflow including the error-handling (retry) path for invalid data.

Sequence Diagram:

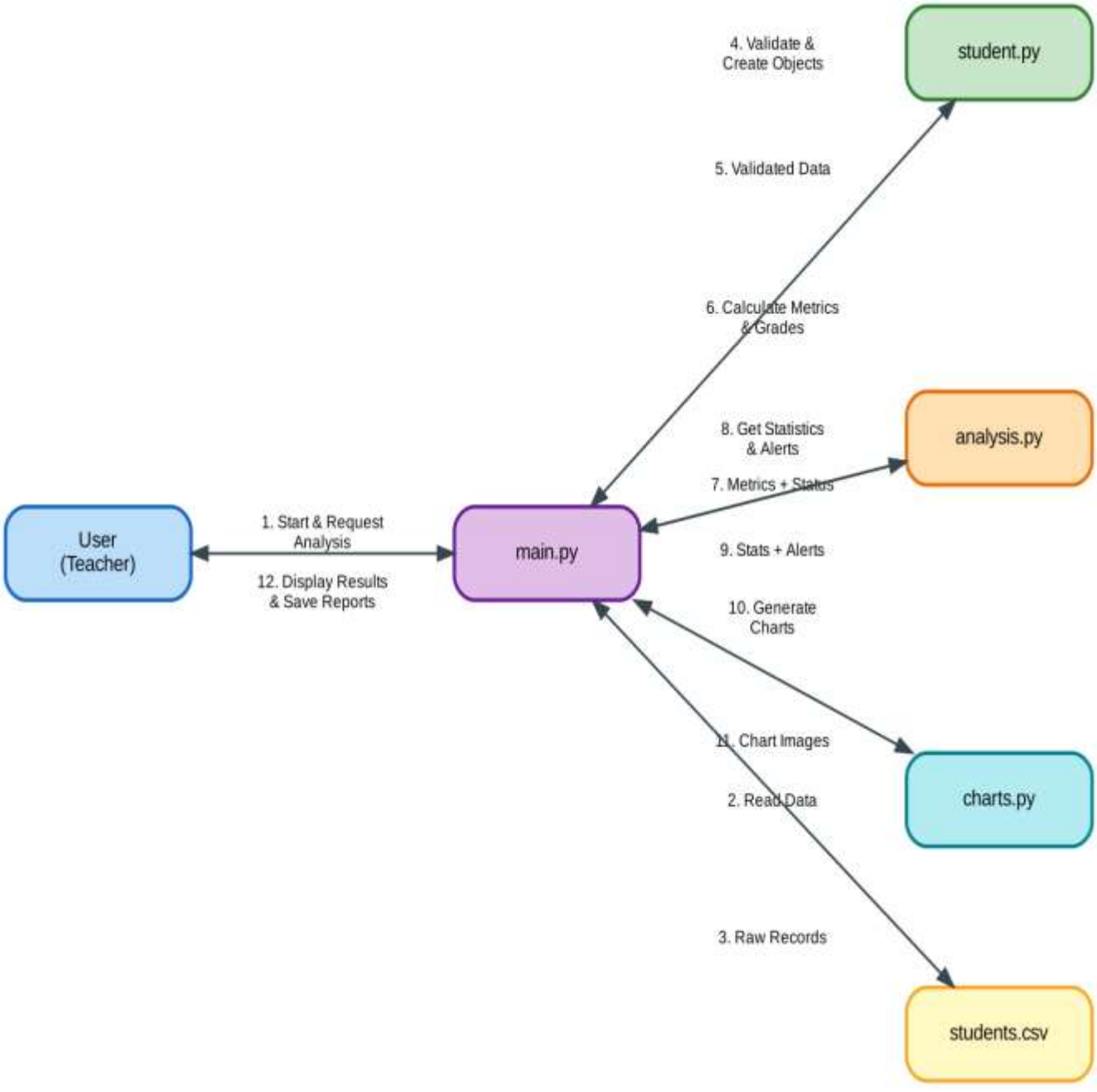


Fig 5.6: Sequence Diagram

The diagram shows the interaction between the User (Teacher), main.py, student.py, analysis.py, charts.py, and the students.csv file.It illustrates the step-by-step message flow starting from the user's analysis request, through data loading, validation, metric calculation, chart generation, and finally returning results.

Flow Chart:

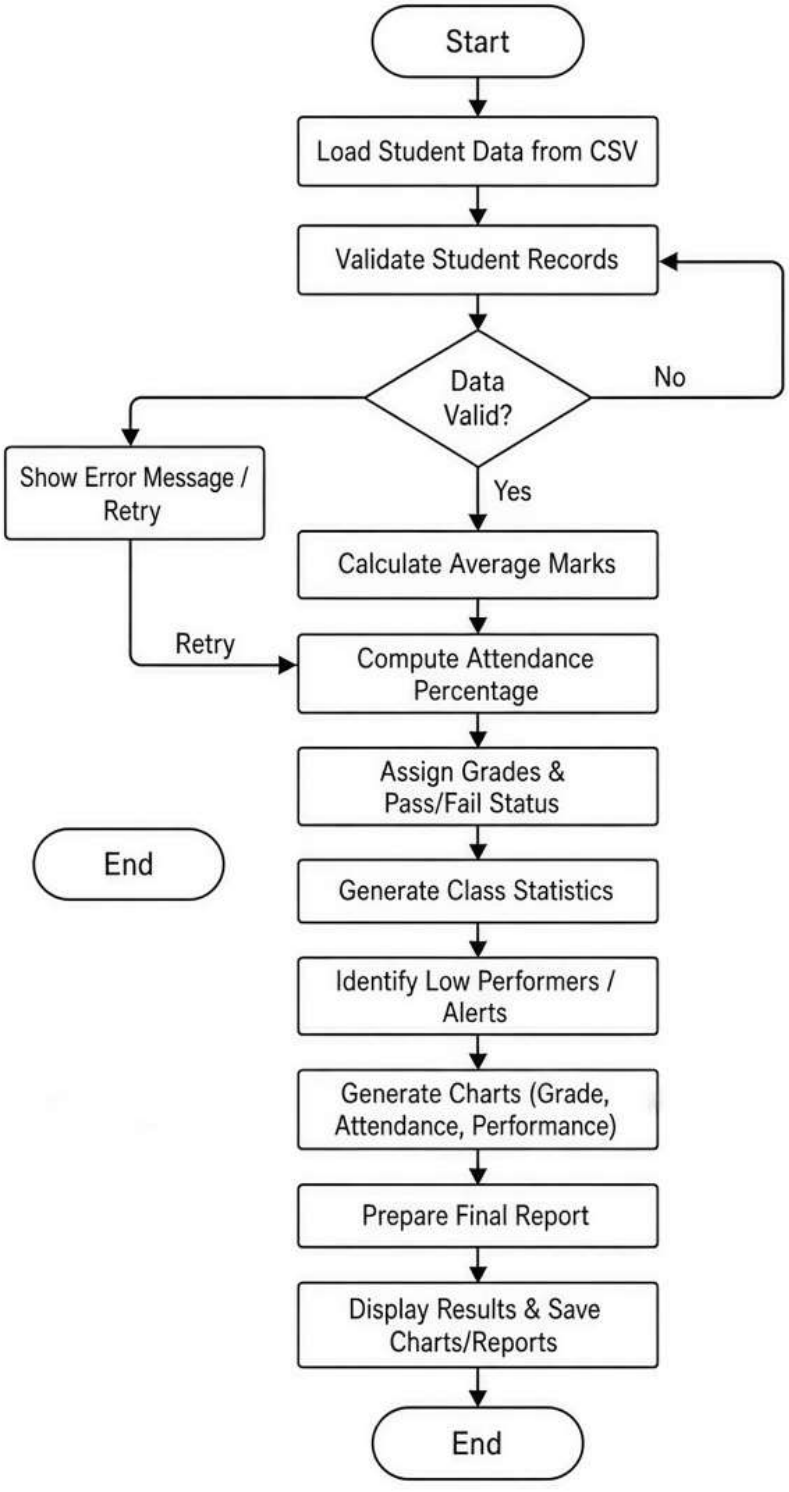


Fig 5.7: Flow Chart

It presents the complete sequential logic of the system using standard flowchart symbols, including decision points for data validation and the path to final report generation. This diagram provides a simple and clear visual representation of the entire program logic.

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the system processes student data and produces analysis results and charts. The method follows a sequential pipeline that begins with CSV input and ends with statistics and visualizations. The overall workflow is shown in the diagrams, and each stage is described in detail.
The application loads student records from student_records.csv, creates structured data objects or DataFrames, computes performance and attendance metrics, filters important cases, and generates charts that are saved to the reports folder. This method allows users to obtain insights from academic data without manual calculations or complex tools.

The process includes the following key stages:
1. Data loading from CSV
2. Data validation and cleaning
3. Performance and attendance analysis
4. Statistical summary generation
5. Chart creation and display/saving
6. Final report presentation

The steps included are –

Step 1: Data Loading

The program reads the student_records.csv file using pandas and prepares a clean dataset for analysis. Missing or invalid values are handled gracefully.

Step 2: Data Preparation

Records are structured so that each student has attributes such as ID, name, marks, and attendance. A clean DataFrame is maintained for further processing.

Step 3: Analysis

The analyzer module (analyzer.py) performs:
- Calculation of average marks
- Attendance percentage computation
- Grade assignment
- Pass/fail determination (based on marks threshold and minimum attendance, commonly 75%)
- Identification of students needing attention
- Class-level aggregates (overall average, highest/lowest performers, attendance statistics)

Step 4: Chart Generation

Functions inside analyzer.py use matplotlib to produce visual representations such as:
- Grade distribution pie/bar charts
- Attendance percentage histograms or bar charts
- Performance comparison plots
- Line graphs and other suitable visualizations
- Charts are saved into the reports directory.

Step 5: Main Program Flow

main.py orchestrates the process: loads data, calls analysis functions from analyzer.py, triggers chart generation, and presents results through a simple menu or sequential execution.

Step 6: Final Output

Users receive console statistics, lists of low-attendance or low-performance students, and generated chart images stored in the reports folder. This supports both verification and decision-making.

This methodology is effective because:

- Data processing is automated and repeatable
- Modular design makes the code easy to understand and extend
- Visualizations improve insight communication
- No external specialized software is required beyond standard Python libraries
- The workflow is suitable for undergraduate-level implementation while demonstrating practical data analysis techniques

REPRESENTATIVE CODE STRUCTURE (based on project files)

- **analyzer.py**: Contains the core functions for loading data, calculating averages, percentages, grades, pass/fail status, summaries, and generating/saving matplotlib charts.
- **main.py**: Entry point that provides the user menu, loads data, invokes analyzer functions, and handles overall program flow.
- **student_records.csv**: Sample dataset used for testing and demonstration.
- **requirements.txt**: Lists required packages (pandas, matplotlib, etc.).
- **reports/**: Directory where generated chart images and any text reports are saved.
- **data/**: Optional supporting data folder.

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how performance and attendance metrics were calculated, how statistics were produced, and how charts were generated. Screenshots of console outputs, charts, and sample reports can be inserted in the respective places.

1. Data Loading Stage

The program loading student_records.csv and displaying the raw dataset in tabular form. Confirms that all student records are correctly imported and ready for processing.

```
==============================================
  STUDENT PERFORMANCE & ATTENDANCE ANALYZER
==============================================
 [1] Add New Student Record
 [2] View All Student Records
 [3] Rank Students by Total Marks (Sorted)
 [4] Identify Low Attendance Defaulters
 [5] Generate Graphical Reports (Bar, Line, Hist)
 [6] Exit
----------------------------------------------
Enter choice (1-6): 2

================================================================================
                    CURRENT STUDENT DATABASE
================================================================================
Roll_No            Name  Math Science English History Computer Total_Marks Average Percentage Grade Classes_Attended Total_Classes Attendance_Pct
    003 Annesha Chowdhury  95.0    85.0    96.0    80.0     88.0       444.0    88.8       88.8     A               45           60          75.0
```

Fig 7.1: Data Loading Stage

2. Performance Analysis

Output showing calculated average marks, assigned grades, and pass/fail results for every student. Also presents the overall class performance summary at the bottom.

```
==============================================
   STUDENT PERFORMANCE & ATTENDANCE ANALYZER
==============================================
 [1] Add New Student Record
 [2] View All Student Records
 [3] Rank Students by Total Marks (Sorted)
 [4] Identify Low Attendance Defaulters
 [5] Generate Graphical Reports (Bar, Line, Hist)
 [6] Exit
----------------------------------------------
Enter choice (1-6): 3

============================================================
            LEADERBOARD (SORTED BY TOTAL MARKS)
============================================================
  Roll_No              Name  Total_Marks  Percentage Grade  Attendance_Pct
0     004            Saikat        456.0        91.2    A+           66.67
1       3  Annesha Chowdhury        444.0        88.8     A           75.00
2     003  Annesha Chowdhury        440.0        88.0     A           83.33

==============================================
```

Fig 7.2: Performance Analysis

3. Attendance Analysis

Display of attendance percentage for each student along with a filtered list of those who have less than 75% attendance. Helps quickly identify students requiring attention.

```
   STUDENT PERFORMANCE & ATTENDANCE ANALYZER
==================================================
 [1] Add New Student Record
 [2] View All Student Records
 [3] Rank Students by Total Marks (Sorted)
 [4] Identify Low Attendance Defaulters
 [5] Generate Graphical Reports (Bar, Line, Hist)
 [6] Exit
--------------------------------------------------
Enter choice (1-6): 4

Enter Minimum Attendance Cutoff (%) [e.g., 75]: 60

============================================================
       STUDENTS WITH ATTENDANCE BELOW 60.0%
============================================================
Great news! All students met or exceeded 60.0% attendance.
```

Fig 7.3: Attendance Analysis

4. Charts Generation

Bar graph comparing the average marks of all students. Makes it easy to visually distinguish top performers from low scorers.

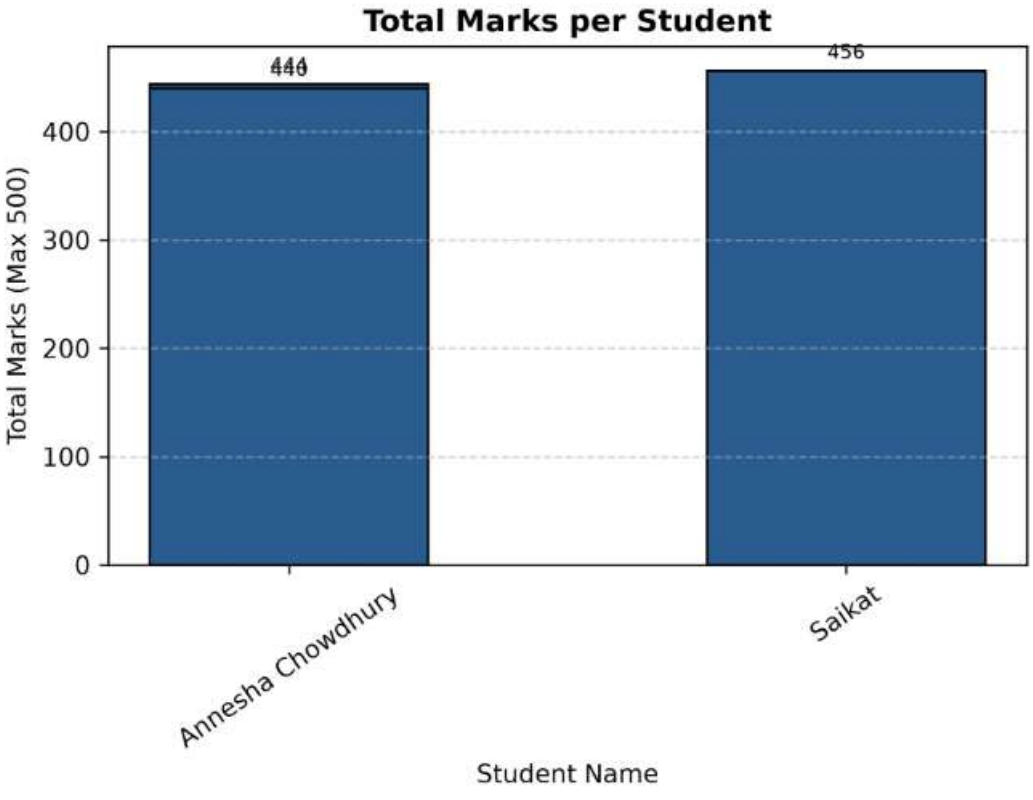
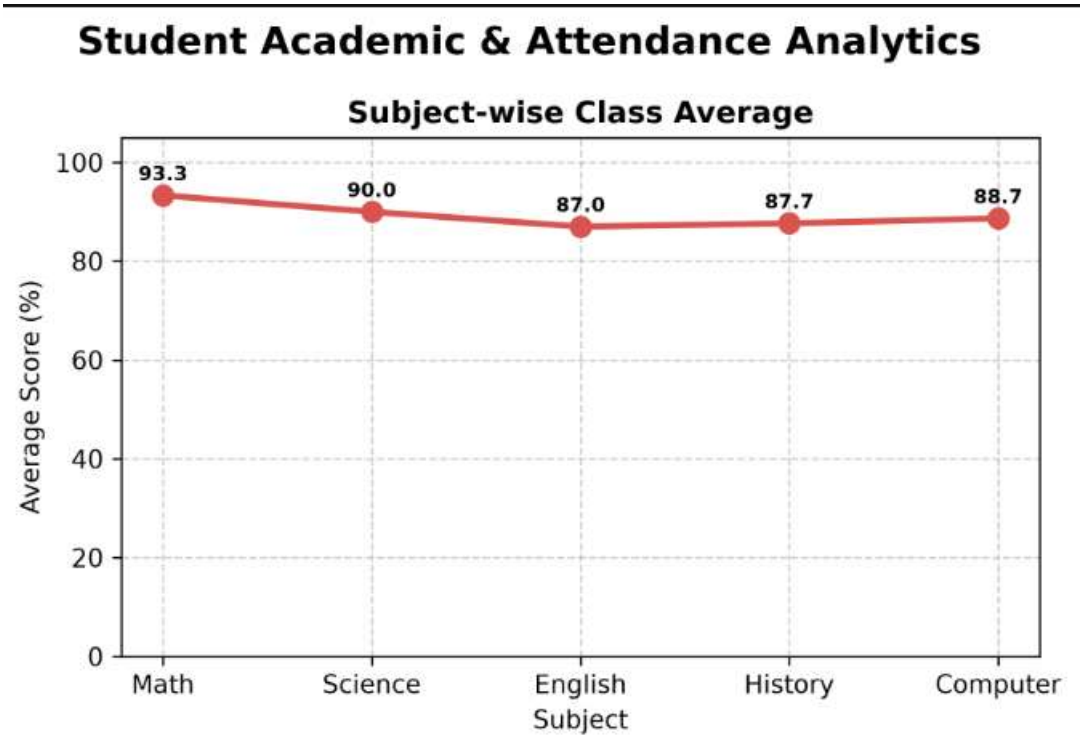
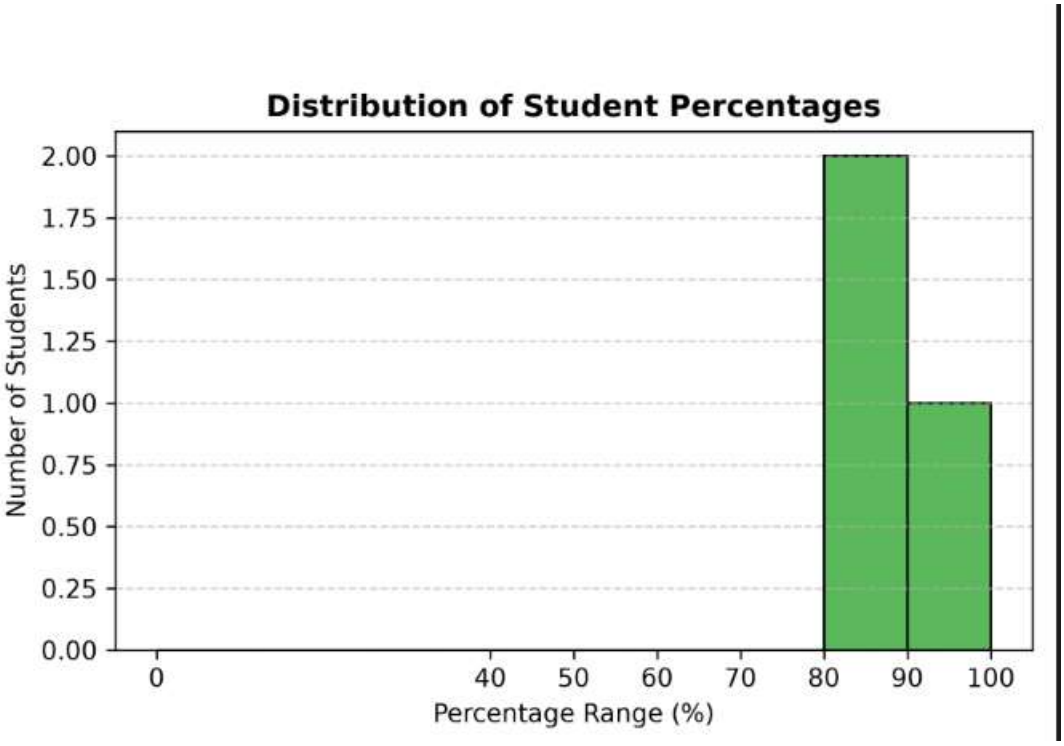


Fig 7.4: Bar Chart Generation



Line chart illustrating the relationship between attendance percentage and average marks. Shows how attendance trends align with academic performance.

Fig 7.5: Line Graph Generation



Histogram representing the frequency distribution of grades across the class. Clearly shows how many students received each grade category.

Fig 7.6: Histogram Generation

CHAPTER 8: FUTURE SCOPE

Several improvements can be added in later versions of the project:

1. **Mobile Application Development**
   Create an Android/iOS mobile app so teachers and students can view performance and attendance reports anytime on their phones.

2. **Role-Based Access System**
   Add separate login for Admin, Teacher, and Student with different levels of access and permissions.

3. **Parent Portal Integration**
   Allow parents to log in and view their child's attendance percentage, marks, and performance reports.

4. **Automated Email/SMS Reports**
   Send weekly or monthly performance and attendance summary reports automatically to students and parents via email or SMS.

5. **Real-time Attendance Marking**
   Integrate biometric devices or QR-code based attendance system for automatic real-time attendance recording.

6. **Dashboard with Filters and Search**
   Build an interactive dashboard where users can filter data by subject, semester, department, or date range.

7. **Performance Comparison Module**
   Add features to compare a student's performance across different semesters or with class average.

CHAPTER 9: CONCLUSION

1. The project successfully demonstrates how a simple yet effective Student Performance & Attendance Analyzer can be built using Python. By combining data handling with pandas and visualization through matplotlib, the system reduces the manual effort required by teachers and provides quick, accurate insights into student performance and attendance patterns.

2. One of the major strengths of this project is its modular structure. Separating the logic into different files (mainly analyzer.py and main.py) makes the code easy to understand, maintain, and upgrade. This design approach also helps beginners learn how large programs can be organized in a clean and professional manner.

3. The system not only calculates important metrics such as average marks, grades, attendance percentage, and pass/fail status, but also presents the results in both textual and graphical formats. These visual charts make it easier for teachers and administrators to identify weak students and take timely corrective actions.

4. Overall, the project achieves its core objective of creating a practical educational tool that is easy to use, requires minimal technical knowledge, and delivers meaningful academic insights. It can serve as a strong foundation for future enhancements such as GUI development, database integration, and predictive analysis, making it a valuable contribution in the field of educational technology.

REFERENCES

1. pandas Development Team. (2024). *pandas: Powerful data structures for data analysis, time series, and statistics*. Available at: https://pandas.pydata.org/

2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. Available at: https://matplotlib.org/

3. Python Software Foundation. (2024). *Python Language Reference, version 3.x*. Available at: https://www.python.org/doc/

4. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.

5. VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.

6. Matplotlib Development Team. (2024). *Matplotlib Documentation*. Available at: https://matplotlib.org/stable/contents.html

7. Project Source Code and Files. Available at: https://1drv.ms/f/c/9A2F6EF399B9BA04/IgAKMcn4Ai-MQIXprnoiYAiOAb_2sxsjeCZezIF4xWjGkEw?e=a9gJmp

8. W3Schools. (2024). *Python Pandas Tutorial*. Available at: https://www.w3schools.com/python/pandas/